

Tab 1

Q1. An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

Answer:

For an e-commerce company storing 10 million product records, a **B+ Tree-based indexed file organization** is recommended.

A B+ Tree allows records to be efficiently searched, inserted, and deleted. Product records can be stored in data blocks, while the B+ Tree index stores search keys such as ProductID.

Advantages:

1. Provides fast searching using ProductID.
2. Supports range queries such as products within a price range.
3. Efficient for large databases.
4. Insertions and deletions can be performed without reorganizing the entire file.
5. B+ Tree remains balanced as the database grows.

Cost Analysis:

- Searching generally requires **$O(\log N)$** operations.
- Insertion generally requires **$O(\log N)$** operations.
- Deletion generally requires **$O(\log N)$** operations.
- Sequential access through leaf nodes is efficient for range queries.

For 10 million records, although maintaining the index requires additional storage, the reduction in search time makes B+ Tree indexing more cost-effective than scanning the entire file.

Conclusion:

A B+ Tree indexed organization is suitable because it provides efficient searching and supports future growth of the e-commerce database.

Q2. A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Answer:

For a banking system, a **composite B+ Tree index** can be created using AccountNumber and TransactionDate.

Example:

```
CREATE INDEX idx_transaction  
ON Transactions(AccountNumber, TransactionDate);
```

The index uses AccountNumber as the first attribute and TransactionDate as the second attribute.

Working:

- Searching for a particular account number is fast.
- Searching for transactions of an account within a date range is also efficient.
- B+ Tree supports both equality and range queries.

Example query:

```
SELECT *  
FROM Transactions  
WHERE AccountNumber = 1001  
AND TransactionDate BETWEEN '2026-01-01' AND '2026-01-31';
```

Index Type:

A **B+ Tree composite index** is appropriate because it efficiently supports both account-number searches and date-range searches.

Conclusion:

The composite B+ Tree index reduces the number of disk blocks that must be accessed and provides fast transaction retrieval.

Q3. A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Answer:

A healthcare system needs indexes on both PatientID and DoctorName.

A **multi-level indexing structure** can be designed as follows:

Primary Index:

- Use PatientID as the primary index.
- A B+ Tree can be used for efficient retrieval.

Secondary Index:

- Create a secondary index on DoctorName.

- Since multiple patients can have the same doctor, the secondary index can contain pointers to multiple patient records.

Example:

```
CREATE INDEX idx_patient_id
ON Patient(PatientID);
```

```
CREATE INDEX idx_doctor_name
ON Patient(DoctorName);
```

Structure:

Level 1: Root Index

↓

Level 2: Intermediate Index

↓

Level 3: Leaf Index

↓

Patient Records

Advantages:

1. Fast retrieval using PatientID.
2. Fast retrieval of all patients associated with a doctor.
3. Multi-level indexing reduces disk accesses.
4. B+ Tree maintains balanced indexing.

Conclusion:

Using a primary B+ Tree index on PatientID and a secondary B+ Tree index on DoctorName provides efficient access through both search attributes.

Q4. A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

Answer:

For a logistics company with frequent insertions and deletions, a **B+ Tree is more suitable**.

Feature	B-Tree	B+ Tree
Data storage	Internal and leaf nodes	Leaf nodes

Range queries	Less efficient	Highly efficient
Sequential access	Moderate	Very efficient
Dynamic insertion	Efficient	Efficient
Dynamic deletion	Efficient	Efficient
Fan-out	Lower	Higher

In a B+ Tree, actual records or record pointers are stored at the leaf level. The leaf nodes are also linked together, making sequential and range searches efficient.

For example, shipments can be searched by:

ShipmentDate BETWEEN Date1 AND Date2

The linked leaf nodes allow the system to move through the required range efficiently.

Conclusion:

A **B+ Tree** is preferred because it supports dynamic insertions and deletions while providing excellent range-query and sequential-access performance.

Q5. Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.

Answer:

For an HR system with 5000 employees, employee records can be accessed using EmployeeID.

A hashing function can be:

$$h(\text{EmployeeID}) = \text{EmployeeID} \text{ MOD } 100$$

This initially creates 100 buckets.

For example:

$$\text{EmployeeID} = 1234$$

$$\begin{aligned} h(1234) &= 1234 \text{ MOD } 100 \\ &= 34 \end{aligned}$$

Therefore, employee 1234 is stored in bucket 34.

Static Hashing

In static hashing, the number of buckets is fixed.

Advantages:

- Simple implementation.
- Fast direct access.
- Suitable when the number of records remains relatively stable.

Disadvantages:

- Overflow can occur when buckets become full.
- Increasing the number of employees may require reorganization.

Extendible Hashing

Extendible hashing dynamically increases or decreases the number of buckets.

Advantages:

- Handles database growth efficiently.
- Reduces overflow.
- Dynamic bucket splitting is possible.
- Suitable for changing employee databases.

Feature	Static Hashing	Extendible Hashing
Bucket size	Fixed	Dynamic
Growth handling	Poor	Excellent
Overflow	More likely	Reduced
Implementation	Simple	More complex

Conclusion:

For an HR system where the number of employees can increase over time, **extendible hashing is preferable** because it dynamically manages database growth.

Q6. An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

Answer:

The airline reservation system requires two types of queries:

1. Point queries using FlightNumber.
2. Range queries using FlightDate.

A suitable solution is to create separate indexes:

```
CREATE INDEX idx_flight_number  
ON Flights(FlightNumber);
```

```
CREATE INDEX idx_flight_date  
ON Flights(FlightDate);
```

A **B+ Tree index** can be used for both attributes.

FlightNumber Index:

FlightNumber
↓
B+ Tree
↓
Flight Record

This provides fast point lookup.

FlightDate Index:

FlightDate
↓
B+ Tree
↓
Flights within Date Range

This is useful for queries such as:

```
SELECT *  
FROM Flights  
WHERE FlightDate BETWEEN '2026-08-01' AND '2026-08-10';
```

Conclusion:

A B+ Tree index on FlightNumber handles point queries efficiently, while another B+ Tree index on FlightDate provides efficient range queries.

Q7. For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

Answer:

For a university student database, a **heap file organization with secondary B+ Tree indexes** can be used.

Student records can be stored as:

Student(StudentID, Name, Department, CGPA)

A secondary index can be created on Department:

```
CREATE INDEX idx_department  
ON Student(Department);
```

Another index can be created on CGPA:

```
CREATE INDEX idx_cgpa  
ON Student(CGPA);
```

The department index provides fast access to students belonging to a particular department.

The CGPA B+ Tree index supports range queries such as:

```
SELECT *  
FROM Student  
WHERE CGPA BETWEEN 8.0 AND 9.0;
```

Advantages:

1. Heap organization allows easy insertion.
2. Department index provides fast department-based searching.
3. CGPA B+ Tree supports range queries.
4. Separate secondary indexes improve query performance.

Conclusion:

Heap file organization combined with secondary B+ Tree indexes on Department and CGPA is an effective solution.

Q8. Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

Answer:

Given:

- Number of records = **100,000**
- Record size = **50 bytes**
- Block size = **4 KB = 4096 bytes**

Step 1: Calculate blocking factor

Blocking Factor = Block Size / Record Size

$$= 4096 / 50$$

$$= 81 \text{ records per block}$$

Therefore, each block can store **81 records**.

Step 2: Calculate required blocks

$$\begin{aligned} \text{Number of Blocks} &= 100,000 / 81 \\ &\approx 1234.57 \end{aligned}$$

Therefore, approximately:

1235 blocks

are required.

Heap File

Records are stored in no particular order.

- Approximately 1235 blocks are required.
- Insertions are fast.
- Searching may require scanning many blocks.

Sorted File

Records are stored in sorted order.

- Approximately 1235 blocks are required for the records.
- Searching is efficient using binary search/indexing.
- Insertions and deletions are more expensive because records may need to be rearranged.

Hashed File

Records are distributed into buckets using a hash function.

- Approximately 1235 blocks are required for the actual records.
- Direct searches using the hash key are very fast.
- Overflow blocks may be required if collisions occur.

Conclusion:

All three organizations require approximately **1235 data blocks**, assuming no significant unused space or overflow. Their major difference is in access performance rather than basic record storage.

Q9. A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

Answer:

For 500 million posts, efficient indexing is essential.

The post table may contain:

Post(PostID, User_ID, Timestamp, Hashtag, Content)

A composite B+ Tree index can be created using:

```
CREATE INDEX idx_user_time  
ON Posts(User_ID, Timestamp);
```

This supports queries such as:

```
SELECT *  
FROM Posts  
WHERE User_ID = 101  
AND Timestamp BETWEEN '2026-08-01' AND '2026-08-23';
```

A separate index can be created for hashtags:

```
CREATE INDEX idx_hashtag_time  
ON Posts(Hashtag, Timestamp);
```

This supports hashtag-based searches and date filtering.

Proposed Strategy

User_ID + Timestamp → Composite B+ Tree

Hashtag + Timestamp → Composite B+ Tree

Advantages:

1. Fast retrieval of a user's posts.
2. Efficient time-based queries.
3. Fast hashtag searches.
4. Composite indexes reduce unnecessary disk accesses.
5. B+ Trees support range queries on timestamps.

For 500 million records, indexes should also be monitored because indexes require additional storage.

Conclusion:

Composite B+ Tree indexes on (User_ID, Timestamp) and (Hashtag, Timestamp) provide efficient access for the major query patterns.

Q10. Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

Answer:

A supermarket billing system generates a large number of transactions every day. Therefore, insertion performance is particularly important.

Operation	Heap File	Sorted File	Hashed File
Insert	Very fast	Slow	Fast
Delete	Moderate	Slow	Fast
Search	Slow	Fast	Very fast for exact search
Range Search	Slow	Fast	Poor
Storage	Simple	Requires ordering	Requires buckets
Best Use	Frequent inserts	Ordered/range queries	Exact-key searches

Heap File

Transactions are stored wherever free space is available.

Advantages:

- Very fast insertion.
- Simple organization.

Disadvantage:

- Searching may require scanning the file.

Sorted File

Records are maintained in sorted order.

Advantages:

- Efficient searching.
- Excellent for range queries.

Disadvantages:

- Insertions can be expensive.
- Deletions may require reorganization.

Hashed File

A hash function determines where each transaction is stored.

For example:

$$h(\text{TransactionID}) = \text{TransactionID} \text{ MOD } N$$

Advantages:

- Very fast exact-match searches.
- Fast insertion and deletion.

Disadvantage:

- Poor performance for range queries.
- Collisions may cause overflow.

Conclusion:

For a supermarket billing system with **1 million daily transactions**, a **hashed file organization** is suitable when transactions are mainly retrieved using an exact transaction ID. A **heap file** is suitable when high-speed insertion is the main requirement, while a **sorted file** is better when frequent range and ordered searches are required.